

The `async` and `await` keywords are used to handle **asynchronous operations** in a clean, readable way—without blocking your program.

◆ The Problem They Solve

Normally, when your program does something slow (like:

- calling an API 🌐
- reading a file 📁
- querying a database 🗄️)

it can **block execution** → everything waits until it finishes.

That's inefficient.

◆ What `async` Does

When you mark a function as `async` :

```
public async Task<string> GetData()
```

👉 It means:

- This function **can run asynchronously**
 - It will **return a Task (or Promise in JS)** instead of a direct value
-

◆ What `await` Does

Inside an `async` function, `await` :

```
var result = await GetDataFromAPI();
```

👉 It means:

- "Pause here **without blocking the thread**"
 - Let other work continue
 - Resume when the result is ready
-

◆ Simple Analogy

Think of it like ordering food 🍔 :

- ❌ Without `async/await`:
You stand at the counter doing nothing until your food is ready.
 - ✅ With `async/await`:
You order → sit down → do something else → get notified when ready.
-

◆ Example (C#)

Without async/await (blocking)

```
var data = GetData(); // waits here (blocking)
Console.WriteLine(data);
```

With async/await (non-blocking)

```
var data = await GetDataAsync();
Console.WriteLine(data);
```
